

Lecture 5

Recurrent Networks: RNNs and LSTMs

Advanced Topics in Artificial Intelligence · UFABC

Part 1 — The Sequence Problem

Lecture roadmap

Part 1 — The Sequence Problem Why dense networks and CNNs fall short

Part 2 — Recurrent Neural Networks (RNN) Hidden state, recurrent equation, unrolling

Part 3 — Training RNNs BPTT, vanishing gradient, gradient clipping

Part 4 — LSTM Memory cell, 4 gates, intuition

Part 5 — GRU and Practical Architectures GRU, bidirectional, stacking, dropout

Part 6 — Code and Applications PyTorch and Keras; classification and seq2seq

Sequential data is everywhere

Text / NLP

"The **cat** that **chased** the mouse **was sleeping** now."

The word "sleeping" only makes sense after understanding "cat" three tokens back.

Time series

Temperature, stock prices, EEG signals.

The value at t depends on values at $t-1, t-2, \dots$

Audio / speech

The current phoneme depends on the preceding phonetic context.

Key property: order matters and sequence length is **variable**.

This breaks the assumptions of dense networks (fixed-size input, no positional awareness) and CNNs (fixed local receptive field).

Why dense networks fail on sequences

Naïve approach: concatenate all tokens and feed to a dense network.

```
"the cat chased the mouse"
```

```
→ [emb_the || emb_cat || emb_chased || emb_the || emb_mouse]  
→ Dense(5 × d → hidden) → Dense(hidden → output)
```

Problem 1 — Variable length

Sentences of 3, 10, 100 tokens → inputs of different sizes.

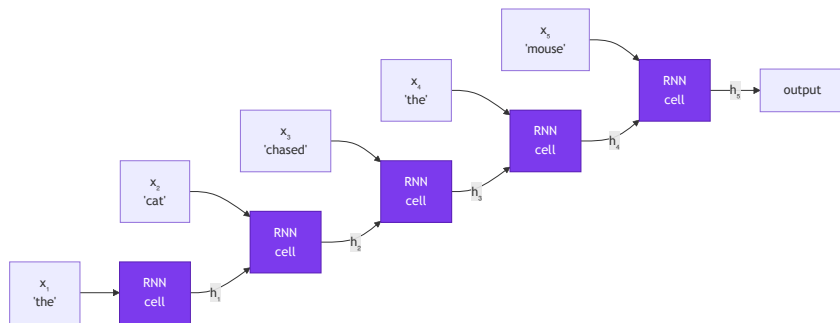
Problem 2 — No weight sharing

The "cat" detector at position 2 uses different weights than at position 7. Doesn't generalize.

Problem 3 — Doesn't scale

What we need

- ✓ Process tokens **one at a time**, in order
- ✓ Maintain an **internal state** that accumulates past information
- ✓ **Share weights** across all positions
- ✓ Work with sequences of **arbitrary length**



Part 2 — Recurrent Neural Networks (RNN)

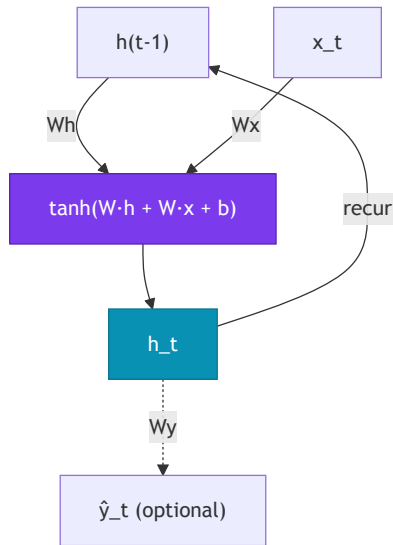
The RNN cell

Core idea: a cell that processes the current token $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, producing a new state $\mathbf{h}_t$.

$$\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$$

- $\mathbf{x}_t \in \mathbb{R}^d$ — token embedding at step t
- $\mathbf{h}_t \in \mathbb{R}^h$ — hidden state at step t (the "memory")
- $W_x \in \mathbb{R}^{h \times d}$ — projects the input
- $W_h \in \mathbb{R}^{h \times h}$ — projects the previous state
- $\mathbf{b} \in \mathbb{R}^h$ — bias
- $\tanh$ — activates and keeps values in $[-1, 1]$

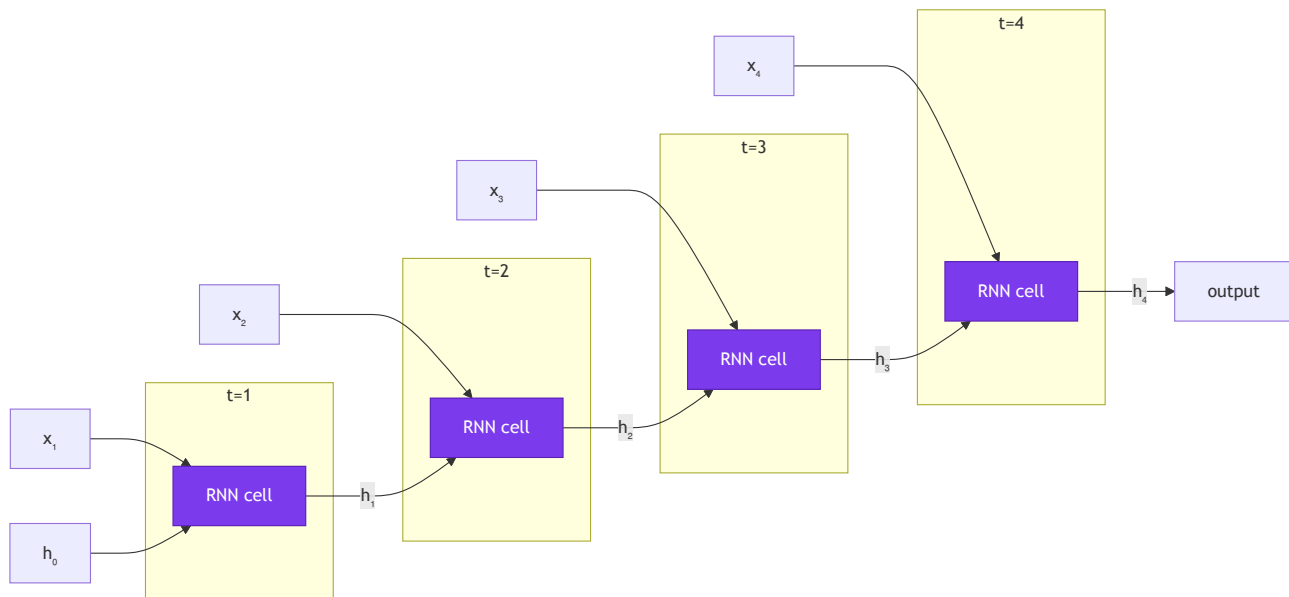
The **same weights** $W_h, W_x, \mathbf{b}$ are used at **every** time step — this is weight sharing.



The loop arrow ($\mathbf{h}_t$ feeds back as $\mathbf{h}_{t-1}$ in the next step) is what defines recurrence.

Unrolling through time

The same cell repeated for each time step:



Shared weights

$W_h, W_x, \mathbf{b}$ are the **same** at every step.

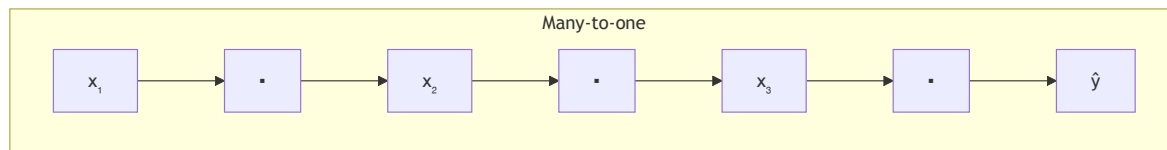
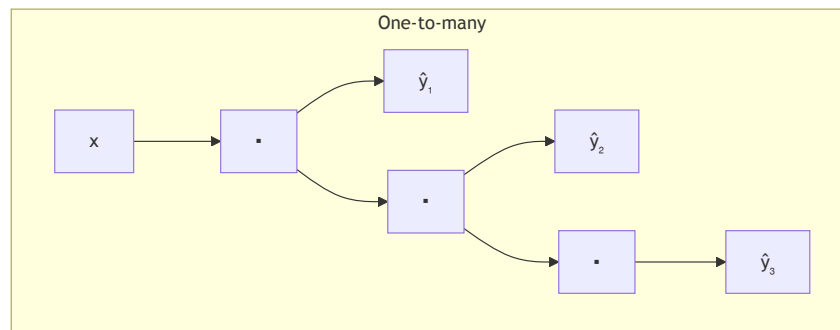
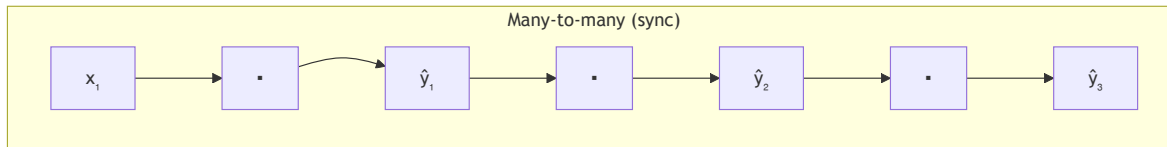
Initial state

$\mathbf{h}_0$ is typically initialized to zeros.

Output

Use $\mathbf{h}_T$ (last state) for classification, or all

Input/output topologies



Many-to-one Sequence $\rightarrow$ single label

One-to-many Single vector $\rightarrow$ sequence

Many-to-many Sequence $\rightarrow$ sequence

Part 3 — Training RNNs

Backpropagation Through Time (BPTT)

The loss gradient must flow back through every time step — through each application of W_h .

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial W_h}$$
$$\frac{\partial \mathcal{L}_t}{\partial W_h} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k}$$

Each factor in the product involves $W_h^\top \cdot \text{diag}(\tanh')$.

The problem:

$$\prod_{k=t}^{T-1} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k} = \prod_{k=t}^{T-1} W_h^\top \cdot \text{diag}(\tanh'(\cdot))$$

Vanishing gradient

If $\|W_h\| < 1$ or $\tanh'$ derivatives are small, the product shrinks **exponentially** → gradient ≈ 0 for long dependencies

Exploding gradient

If $\|W_h\| > 1$, the product grows **exponentially** → NaN / training divergence

Vanishing gradient — intuition

Imagine a chain of multiplications with factor $\gamma < 1$ at each step:

```
T = 10 steps,  $\gamma = 0.9$ :  
gradient  $\leftarrow 0.9^{10} \approx 0.35$  (still ok)  
  
T = 50 steps,  $\gamma = 0.9$ :  
gradient  $\leftarrow 0.9^{50} \approx 0.005$  (nearly zero)  
  
T = 100 steps,  $\gamma = 0.9$ :  
gradient  $\leftarrow 0.9^{100} \approx 2 \times 10^{-5}$  (effectively zero)
```

The network "forgets" what happened more than ~10–20 steps back.

Practical consequence

- ✗ Vanilla RNNs cannot learn long-range dependencies
- ✗ On tasks like translation or long-document analysis, performance degrades

Partial fix: gradient clipping

```
# PyTorch — clip gradients before update  
torch.nn.utils.clip_grad_norm_(  
    model.parameters(), max_norm=1.0  
)
```

- ✓ Gradient clipping fixes *exploding gradients*
- ✗ But it does **not** fix vanishing — we need a different architecture:

Part 4 — Long Short-Term Memory (LSTM)

Motivation — explicit memory

Hochreiter & Schmidhuber (1997)

Core idea: separate "long-term memory" from the "working state".

Vanilla RNN:

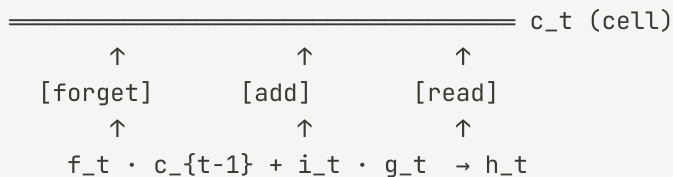
A single vector $\mathbf{h}_t$ must do everything — accumulate history, filter noise, carry useful information.

LSTM:

Two distinct vectors:

- $\mathbf{c}_t$ — **memory cell** (long-term, smooth flow)
- $\mathbf{h}_t$ — **hidden state** (short-term, filtered output)

The conveyor belt metaphor



The cell $\mathbf{c}$ is like a **conveyor belt** that carries information through time with minimal perturbation — the gates decide what enters, what exits, and what gets erased.

- $\mathbf{f}_t$ (*forget gate*) — how much of $\mathbf{c}_t$ to keep (0 = erase, 1 = preserve)
- $\mathbf{i}_t$ (*input gate*) — how much of the candidate $\mathbf{g}_t$ to write into the cell
- $\mathbf{g}_t$ (*candidate*) — proposed new content, generated by $\tanh$
- $\mathbf{h}_t$ — output filtered by the *output gate* from $\mathbf{c}_t$

LSTM — Equation 1: Forget Gate

$$\mathbf{f}_t = \sigma(W_f [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f)$$

"The past filter"

- $\sigma(\cdot) \rightarrow$ output in **(0, 1)** per dimension independently
- $\mathbf{f}_t \approx \mathbf{0} \rightarrow$ erases that dimension from $c(t-1)$
- $\mathbf{f}_t \approx \mathbf{1} \rightarrow$ preserves that dimension intact
- Learns what to discard from $\mathbf{h}(t-1)$ and $\mathbf{x}_t$

Example: when processing "Peter" after "John went to the market", $\mathbf{f}_t \approx \mathbf{0}$ in the active-subject dimension **erases "John"** to make room for the new referent.

Gradient: $\partial c_t / \partial c(t-1) = \mathbf{f}_t$. When $\mathbf{f}_t \approx \mathbf{1}$, the gradient flows back **without attenuation** — the core solution to the *vanishing gradient*.

LSTM — Equation 2: Input Gate + Candidate

$$\mathbf{i}_t = \sigma(W_i [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad \tilde{\mathbf{c}}_t = \tanh(W_c [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c)$$

"What" and "how much" to write

Two distinct roles working together:

- $\hat{\mathbf{c}}_t$ (tanh) — proposes the *content*: values in $(-1, 1)$, encodes polarity/direction
- $\mathbf{i}_t$ (σ) — the *gatekeeper*: decides how much of $\hat{\mathbf{c}}_t$ actually enters the cell
- Product: $\mathbf{i}_t \odot \hat{\mathbf{c}}_t$ = filtered new information

Example: when processing "Peter", $\hat{\mathbf{c}}_t$ encodes properties of the new subject; $\mathbf{i}_t$ decides which cell dimensions to write to.

Why tanh for $\hat{\mathbf{c}}_t$? Memory needs to represent *direction* — increment or decrement a feature. σ only gives positive values; tanh allows encoding polarity.

LSTM — Equation 3: Cell Update

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{erase the old}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{add the new}}$$

The "memory conveyor belt"

$\mathbf{c}_t$ is an **additive highway** carrying information over time:

- $\mathbf{f}_t \odot \mathbf{c}_{t-1} \rightarrow$ keeps what still matters
- $\mathbf{i}_t \odot \tilde{\mathbf{c}}_t \rightarrow$ adds new content
- $\odot$ = Hadamard (element-wise) product

Extreme cases: $\mathbf{f}_t \approx 1, \mathbf{i}_t \approx 0 \rightarrow$ cell **frozen** · $\mathbf{f}_t \approx 0, \mathbf{i}_t \approx 1 \rightarrow$ cell **reset**

Why does addition solve the vanishing gradient?

In a vanilla RNN the gradient flows through $W_h \cdot \tanh'(\cdot)$ and shrinks exponentially at every step.

In the LSTM: $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathbf{f}_t$. When $\mathbf{f}_t \approx 1$, the gradient flows back **directly**, bypassing non-linearities — the addition creates a shortcut in backpropagation.

LSTM — Equation 4: Output Gate

$$\mathbf{o}_t = \sigma(W_o [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

"Private memory vs. public output"

- $\mathbf{c}_t$ = internal memory (can store much more than is exposed)
- $\mathbf{o}_t$ (σ) = decides which dimensions of $\mathbf{c}_t$ "leak" into $\mathbf{h}_t$
- $\tanh(\mathbf{c}_t)$ = normalises the cell to $(-1, 1)$
- $\mathbf{h}_t$ = visible output — goes to the next layer *and* to $t+1$

Example: in translation, $\mathbf{c}_t$ stores a noun's grammatical gender for several steps. The output gate exposes it only when the model is about to generate an adjective requiring agreement.

Summary: forget → erase $\mathbf{c}(t-1)$ · input → write to $\mathbf{c}_t$ · output → expose $\mathbf{h}_t$

Gate intuition — step by step

Example: "Mary went to the market. She bought apples." — the network needs to know "She" = "Mary".

Forget gate

When processing "She", the network can use $\mathbf{f}_t \approx 1$ to **retain** "Mary" in the cell, or $\mathbf{f}_t \approx 0$ to **erase** irrelevant information from previous steps.

$$\mathbf{c}_t \leftarrow \mathbf{f}_t \odot \mathbf{c}_{t-1}$$

Input gate

When seeing "She", $\mathbf{i}_t$ decides how much of the candidate $\tilde{\mathbf{c}}_t$ (encoding the pronoun and its possible referent) should be **written** to the cell.

$$\mathbf{c}_t += \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Output gate

When generating the next word, $\mathbf{o}_t$ controls **how much** of "Mary"'s memory is passed into $\mathbf{h}_t$ and therefore influences the prediction.

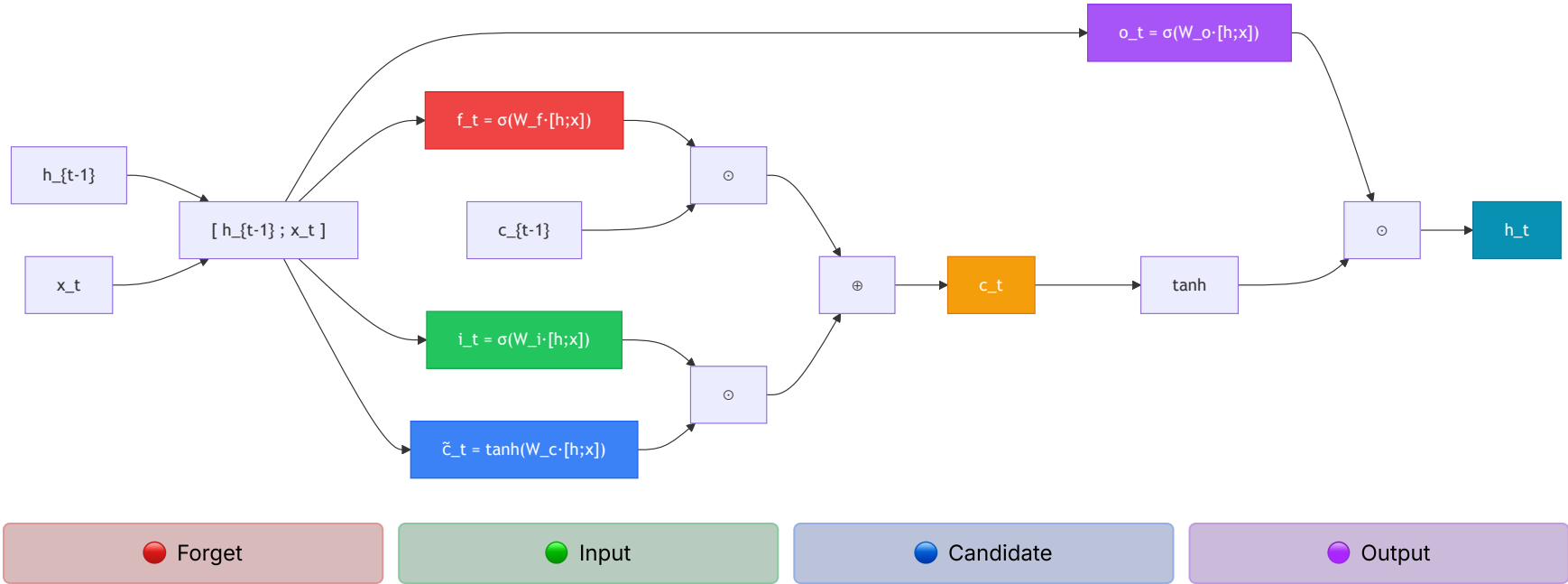
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Why does LSTM solve vanishing gradients?

The gradient path through the cell $\mathbf{c}$ is **additive**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \dots$

Addition doesn't multiply the gradient — it can flow back many steps without shrinking exponentially (as long as $\mathbf{f}_t \approx 1$).

Full LSTM cell diagram



Part 5 — GRU and Practical Architectures

GRU — Gated Recurrent Unit

Cho et al. (2014) — simplified LSTM with only **2 gates** and **no separate cell**.

Reset gate

$$\mathbf{r}_t = \sigma(W_r [\mathbf{h}_{t-1}; \mathbf{x}_t])$$

Controls how much of the previous state influences the candidate.

Update gate

$$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}; \mathbf{x}_t])$$

Candidate + final update

$$\tilde{\mathbf{h}}_t = \tanh(W [\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t])$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

GRU vs LSTM

	LSTM	GRU
Gates	3 (f, i, o)	2 (r, z)
State	$\mathbf{h}_t + \mathbf{c}_t$	only $\mathbf{h}_t$
Parameters	More	Fewer
Training	Slower	Faster
Performance	Slightly better for very long sequences	Comparable on most tasks

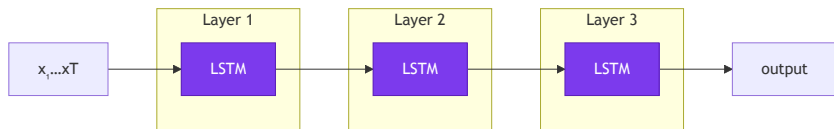
Practical rule: start with GRU if sequences are not extremely long or the dataset is small. Use LSTM when you need very long-term

Bidirectional RNN

Stacked RNNs and Dropout

Stacking

The output of one RNN layer feeds the next — each layer learns more abstract representations.



2–4 layers are common; more than that rarely helps and increases the risk of overfitting.

Dropout in RNNs

Standard dropout between layers works, but **not** inside the recurrent cell (it would destroy memory).

Variational dropout (Gal & Ghahramani, 2016): applies the **same mask** at all time steps.

```
# PyTorch – dropout between LSTM layers
nn.LSTM(
    input_size=128,
    hidden_size=256,
    num_layers=3,
    dropout=0.3,      # applied between layers
    bidirectional=False
)
```

The `dropout` argument of `nn.LSTM` is only applied between intermediate layers — the last layer has no automatic dropout.

Part 6 — Code and Applications

nn.LSTM and nn.GRU — PyTorch

```
import torch
import torch.nn as nn

# Sequence: (batch=2, seq_len=10, input_size=64)
x = torch.randn(2, 10, 64)

# LSTM
lstm = nn.LSTM(
    input_size=64,
    hidden_size=128,
    num_layers=2,
    batch_first=True,    # (batch, seq, feat)
    dropout=0.2,
    bidirectional=False
)

# Forward pass
# h0, c0: (num_layers, batch, hidden)
h0 = torch.zeros(2, 2, 128)
c0 = torch.zeros(2, 2, 128)

output, (hn, cn) = lstm(x, (h0, c0))
# output: (2, 10, 128) - all h_t
```

```
# GRU - same interface, no c
gru = nn.GRU(
    input_size=64,
    hidden_size=128,
    num_layers=2,
    batch_first=True,
    bidirectional=True  # hidden_size × 2 in output
)

output, hn = gru(x)
# output: (2, 10, 256) - bidirectional → ×2
# hn:      (4, 2, 128) - 2 layers × 2 directions

# Classification: use last token
last_hidden = output[:, -1, :] # (2, 256)

# Or mean over the sequence
mean_hidden = output.mean(dim=1) # (2, 256)
```

With `batch_first=False` (default): format `(seq, batch, feat)`. I recommend always using `batch_first=True` for consistency with

Sentiment classification — PyTorch

```
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, num_classes):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                              batch_first=True, dropout=0.3)
        self.dropout = nn.Dropout(0.3)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        # x: (batch, seq_len)
        emb = self.embed(x)           # (batch, seq_len, embed_dim)
        out, (hn, _) = self.lstm(emb) # out: (batch, seq_len, hidden)
        # Use last hidden state of last layer
        last = hn[-1]                 # (batch, hidden_dim)
        last = self.dropout(last)
        return self.fc(last)          # (batch, num_classes) - logits

model = SentimentLSTM(10_000, 128, 256, 2, 2) # binary: pos/neg
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    for tokens, labels in train_loader:
        logits = model(tokens)
```

LSTM in Keras

```
import tensorflow as tf
from tensorflow import keras

# Full pipeline
vectorizer = keras.layers.TextVectorization(
    max_tokens=10_000,
    output_sequence_length=128
)
vectorizer.adapt(train_texts)

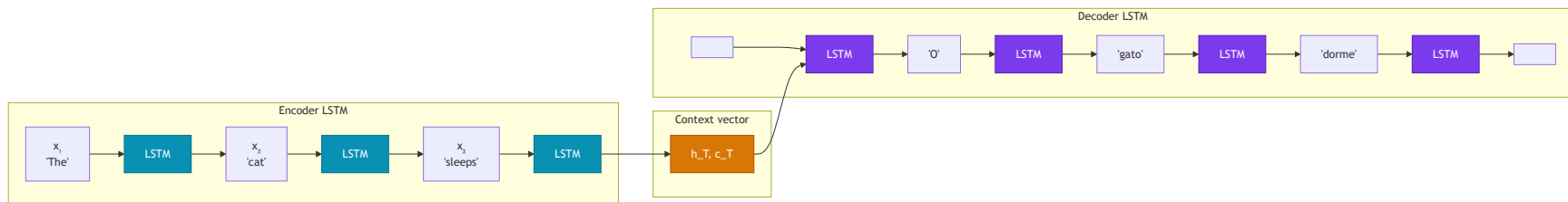
model = keras.Sequential([
    keras.Input(shape=(1,), dtype='string'),
    vectorizer,
    keras.layers.Embedding(10_000, 128, mask_zero=True),
    # Stacked bidirectional LSTM
    keras.layers.Bidirectional(
        keras.layers.LSTM(128, return_sequences=True,
                           dropout=0.2, recurrent_dropout=0
        ),
    ),
    keras.layers.Bidirectional(
        keras.layers.LSTM(64, dropout=0.2)
    ),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.3),
```

```
model.compile(
    loss='binary_crossentropy',
    optimizer=keras.optimizers.Adam(1e-3),
    metrics=['accuracy']
)
model.summary()

# Model: "sequential"
# -----
# Layer (type)           Output Shape          Param #
# =====
# text_vectorization      (None, 128)           0
# embedding                (None, 128, 128)     1,280,000
# bidirectional            (None, 128, 256)     263,168
# bidirectional_1         (None, 128)          98,816
# dense                    (None, 64)           8,256
# dropout                  (None, 64)           0
# dense_1                  (None, 1)            65
# =====

model.fit(train_texts, train_labels,
          validation_split=0.2, epochs=10,
          batch_size=64)
```

Application: sequence-to-sequence (seq2seq)



Encoder: processes the input sequence, compresses it into a context vector (h_T, c_T) .

Decoder: initialized with (h_T, c_T) , generates the output token by token autoregressively.

Limitation of vanilla seq2seq: the context vector is a bottleneck — compressing an entire sequence into a single vector loses information. The solution is the **attention mechanism** (→ Lecture 6: Transformers).

Comparison: RNN, LSTM, GRU

	Vanilla RNN	LSTM	GRU
Memory	State $\mathbf{h}_t$	$\mathbf{h}_t + \mathbf{c}_t$	only $\mathbf{h}_t$
Gates	—	3 (f, i, o)	2 (r, z)
Parameters	Baseline	4× more than RNN	3× more than RNN
Long dependencies	✗ Vanishing	✓ Good	✓ Good
Speed	Fast	Slower	Intermediate
Typical use	Quick baseline	NLP standard until 2018	Efficient alternative

In **2017–2018**, LSTMs were the state of the art in translation, summarization and speech recognition. With the arrival of **Transformers** (Vaswani et al., 2017), they were gradually superseded — but RNNs/LSTMs remain useful for low-latency inference, memory-constrained devices and time-

Summary

The problem

- Sequential data has order and variable length
- Dense networks: no order, no weight sharing
- CNNs: limited receptive field

Vanilla RNN

- $\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t)$
- Weights shared across all steps
- Suffers from vanishing gradient

BPTT and gradients

- Gradient flows back through chained multiplications
- Vanishing: $\gamma^T \rightarrow 0$ with $\gamma < 1$
- Exploding: gradient clipping

LSTM

- Cell $\mathbf{c}_t$ + gates $\mathbf{f}$, $\mathbf{i}$, $\mathbf{o}$
- Additive path: solves vanishing
- $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

GRU and practices

- 2 gates, more efficient than LSTM
- Bidirectional: past + future context
- Stacking + variational dropout

Next lecture

- Context vector bottleneck
- Attention mechanism
- Full Transformer architecture

Next lecture

Lecture 6 — Transformers

- Seq2seq bottleneck as motivation
- Self-attention mechanism
- Scaled Dot-Product Attention and Multi-Head
- Positional encoding
- Full architecture (encoder + decoder)
- BERT and GPT: fine-tuning and generation

For this week

- Notebook `lec05-rnn.ipynb` :
 - Sentiment classification with LSTM (IMDB)
 - RNN vs LSTM vs GRU comparison
 - Learning curves and gradient analysis
 - Hidden state visualization with t-SNE

Thank you! Questions?

CCM-109 · Deep Learning · UFABC